

# MedAssistAI - Medical Symptom Analysis & Diseases Prediction System

## Individual Contribution Report

| Field            | Details                                        |
|------------------|------------------------------------------------|
| Name             | Koritala Lakshmi Sahitya                       |
| Role             | Full Stack Developer                           |
| Evaluation Phase | Milestone - 3                                  |
| Status           | ML Integration & Full-Stack System Integration |

### 1. Overview

During Milestone 3, the developed MedAssist AI frontend and backend were integrated with the trained machine learning models and healthcare workflows. The main focus was on connecting ML prediction functionality with the existing web application, integrating backend APIs, enabling inter-role communication, and improving the overall functionality of the healthcare system.

### 2. ML Model Integration

- Integrated the trained machine learning models into the Flask backend.
- Added the required trained model files to the project.
- Configured the backend to load the models during application startup.
- Connected the ML models with the existing frontend modules.
- Established the communication between user input, backend APIs, ML models, and prediction results.

### 3. AI Symptom Analysis Integration

- Connected the **AI Symptom Analysis** page with the Flask backend.
- Integrated the trained disease prediction model with the symptom input workflow.
- Implemented the flow:

**Patient Symptoms → Frontend → Flask API → ML Model → Prediction → Frontend**

- User-selected symptoms are sent to the backend after clicking **Analyze**.
- The model processes the input and returns the prediction.
- The prediction result is displayed in the existing **Analysis Results** section.

- Added appropriate validation and error handling for incomplete or invalid inputs.

#### 4. Medical Report Analysis

- Developed the workflow for analyzing uploaded medical reports.
- Added medical-document validation before generating results.
- Ensured that irrelevant/non-medical documents do not generate medical predictions.
- Connected report processing with the backend.
- Structured the system to extract relevant medical information from valid reports.
- Integrated the analysis results into the existing Medical Report Analysis interface.

#### 5. Backend API Integration

- Connected frontend modules with the Flask backend through APIs.
- Added/updated backend endpoints required for:
  - Disease prediction
  - Symptom analysis
  - Medical report processing
  - File uploads
  - Authentication
  - Healthcare data exchange
- Implemented request validation and response handling.
- Maintained the existing backend functionality while adding the required integrations.

#### 6. Inter-Role Data Integration

Implemented the necessary data flow between healthcare roles.

Examples:

- **Patient** → **Doctor**: Symptoms, reports, prediction results and relevant medical information.
- **Lab Assistant** → **Doctor**: Uploaded laboratory reports and test results.
- **Doctor** → **Patient**: Prescriptions and relevant medical information.
- **Receptionist** → **Doctor/Patient**: Appointment details and schedules.
- **Pharmacy** → **Doctor/Patient**: Prescription and medication-related information.

This allows information entered in one role to be available to the appropriate role when required.

#### 7. Authentication & Role-Based Access

- Integrated and tested role-based authentication.
- Maintained separate access for:

- Patient
- Doctor
- Lab Assistant
- Receptionist
- Pharmacy
- Hospital Admin
- Super Admin
- Implemented demo accounts for easier project demonstration and testing.
- Verified role-based dashboard redirection and access permissions.
- Preserved the existing OTP-based authentication for normal users.

## 8. API & External Service Integration

Integrated the required external services into the application, including:

- **Gemini API** for chatbot/AI-assisted functionality.
- **Location services** for nearby hospital identification.
- **OpenStreetMap-based services** for location and hospital search.
- Application-wide language/translation functionality.

The integrations were connected without changing the existing application design.

## 9. Application-Wide Language Support

- Extended the multilingual framework across the web application.
- Structured the translation system for common UI elements, including:
  - Navigation
  - Buttons
  - Forms
  - Dashboards
  - Messages
  - Role-specific modules
- Added support for major Indian and commonly used international languages.
- Maintained a centralized language-selection mechanism.

## 10. File Upload & Report Workflow

Improved file handling for healthcare documents.

Implemented/structured:

- Medical report upload
- Laboratory report upload
- PDF/image validation
- Backend file processing
- Report storage
- Report sharing with relevant healthcare roles

This enables reports uploaded by one role to be accessed by other authorized roles when required.

## **11. Error Handling & Validation**

Improved application reliability by handling:

- Invalid user inputs
- Invalid file formats
- Non-medical documents
- Missing required fields
- Backend API failures
- ML prediction errors
- Unauthorized role access
- Invalid authentication/role information

The objective was to ensure that errors are handled gracefully without affecting other application modules.

## **12. Frontend - Backend - ML Workflow**

The overall architecture was connected as:

**React/Vite Frontend - Flask Backend - ML Model/API - Prediction/Analysis - Frontend**

This integration allows the trained ML models to function as part of the actual MedAssist AI web application rather than operating separately.

## **13. Testing & Integration**

- Tested different role-based workflows.
- Verified frontend and backend communication.
- Tested ML prediction integration with user inputs.
- Tested file-upload workflows.
- Checked inter-role data sharing.
- Identified and resolved integration and authorization issues.

- Ensured existing functionality was preserved while adding new integrations.

## 14. Current Status

The major components of MedAssist AI are now integrated into a common full-stack workflow.

The application currently contains:

- Role-based healthcare dashboards
- Authentication
- AI/ML prediction integration
- Symptom analysis
- Medical report analysis
- Healthcare report management
- Appointment workflows
- Inter-role data sharing
- External API integrations
- Multilingual support
- Backend services using Flask

## 15. Next Steps

- Complete final end-to-end testing of all roles.
- Validate ML predictions with different inputs.
- Complete deployment configuration.
- Test APIs and ML models in the deployed environment.
- Verify complete data synchronization between roles.
- Perform final bug fixing and performance testing.
- Prepare the application for final demonstration.

## 16. Conclusion

Milestone 3 focused on **integrating the trained AI/ML models with the existing full-stack MedAssist AI application**. The frontend, Flask backend, ML prediction pipeline, APIs, authentication, healthcare workflows, and inter-role communication were brought together to create a functional AI-powered healthcare system. The application is now progressing toward final testing, deployment, and demonstration.